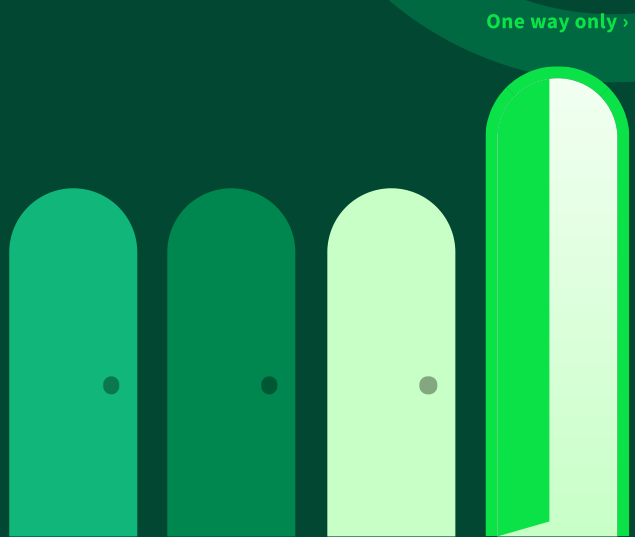
GitHub Copilot agent skill · Technical design

One-Way Door

Technical design for a GitHub Copilot skill that checks every change for a way back, before it is merged.

Know before you merge ›



About this document

This is the technical design for **One-Way Door**, a GitHub Copilot agent skill. It covers the problem, the user story, the skill and how it works, where the code lives, how to set it up and run the demo, what changes for the user, how to reuse it, its known limitations and the guardrails it keeps to. Everything described is built and tested.

Document	**Technical design document (TDD)**
Version	**1.0, draft for hackathon review**
Date	**13 October 2026**
Owner	**[Team name]**
Repository	**TBD**
Audience	**Engineers, reviewers, platform and release teams, judges**

At a glance (measured)

4 pass · 1 fail
rollback matrix, as the skill predicts

24 patterns
one-way doors in the catalogue, each with a safe split

62 + 6 tests
Node tests (skill, hook, plugin) and Control Centre unit tests

Contents
Ten sections
Page 2 ›

01–02
Problem and story
Page 3 ›

03–06
Skill, setup, demo
Page 5 ›

07–10
Outcomes and limits
Page 18 ›

Contents

What this document covers

01	**Problem statement**	Why rollback fails, and why today's checks miss it	3
02	**User story**	Who it is for, the stories and the acceptance criteria	4
03	**Skill name and mission**	What the skill is, what it does, and how it works	5
	3.1 Name, mission and capabilities		5
	3.2 Components and design principles		6
	3.3 How it works		7
04	**Repository link**	Where the code lives (TBD), its layout, branches and quality gates	8
05	**Setup instructions**	The tech stack and installing it in VS Code, CLI, cloud agent and code review	9
	5.1 Tech stack and install options		9
	5.2 Setup by Copilot surface		10
06	**Demo instructions**	Architecture, running the live demo on a laptop, and its screens	11
	6.1 Set up and start		11
	6.2 Architecture: the demo system		12
	6.3 Architecture: the demo runtime		13
	6.4 Run the five acts		14
	6.5 Screens: the PR and the rollback		15
	6.6 Screens: £0.00, and the review		16
	6.7 Screens: the proof		17
07	**Before and after completions**	What Copilot says without the skill and with it, and what changes in the system	18
08	**Reuse statements**	What can be reused as it is, what to configure, and how to extend it	19
09	**Known limitations**	What it doesn't do yet, the impact, and the workaround	20
10	**Guardrails**	What the skill, scripts and hook will and won't do	21

Document control

Version	Date	Change
1.0	14 Oct 2026	First issue: skill, hook, plugin, laptop demo, diagrams and screens

Related documents

- `README.md`: overview and results
- `docs/setup.md`: full setup guide, with sources
- `DEMO.md` and `docs/demo-script.md`: running and presenting the demo
- `docs/copilot-formats.md`: verified Copilot formats
- `One-Way-Door-proposal.pdf`: the hackathon proposal

Terms used

One-way door	A change that leaves state the previous release can't handle, so rollback doesn't restore service.
Two-way door	A change that can be deployed and rolled back safely.
Door closes	The deploy moment after which rollback is no longer safe.
Rollback window	How long after deploy a rollback is still safe.
Blast radius	Other code that reads the same table, column, topic or field.
The split	Expand → migrate → contract → drop: the same change in steps that each roll back.
Surface	Where Copilot runs: VS Code, Copilot CLI, cloud agent, code review.

01 · Problem statement

Rollback is the safety net nobody tests

Problem statement

Engineers can't tell, before they merge, whether the previous release can still run against the state the new release leaves behind. Tests, CI and code review all check new code against new state. When the old code can't cope, rollback fails loudly, or worse, appears to work while it quietly corrupts data.

Where one-way doors hide (the four buckets the skill checks)

Data
Columns renamed, dropped, retyped or converted in place. Migrations edited after they ran.

Messages
Event fields removed, renamed or retyped. Consumers that read a missing number as zero.

State
Cache entries and idempotency keys rewritten in a format the old code can't read.

External
Webhook payloads sent, queues deleted, API versions removed. Redeploying can't recall them.

Why today's checks miss it

1. **Tests run new code against new state.**
   Rollback creates the opposite combination, old code against new state, and nothing exercises it.
2. **Migrations only go forward.**
   Flyway and its peers have no undo. Going back means a hand-written script under pressure.
3. **Reviewers see the diff, not the readers.**
   The job that breaks is often not in the PR at all.
4. **The worst failures are silent.**
   A missing field becomes 0.0; no exception, nothing in the dead-letter queue.

Evidence: the £0.00 settlement, reproduced

- **The change:** PR #482 moves money from `double` pounds to `long` pence. Migration V3 renames and converts the column in place. Tests are green.
- **The rollback:** the old ledger won't start (`missing column [amount]`). After a hand hotfix it books the queued events.
- **The result** (recorded run, 13 Oct 2026): **50 settlements booked at £0.00, £12,708.11 missing**, dead-letter topic empty, every service healthy. Finance's reconciliation, not in the PR, broke first.

Scope

In scope

- Reviewing a pull request or diff for rollback safety, before merge, inside GitHub Copilot.
- Database migrations, event schemas, caches, idempotency keys and external contracts.
- Finding other readers in the same repository, and proposing a safe split.
- Blocking agent edits to migrations that have already shipped.

Out of scope (this version)

- Readers in other repositories (planned: see section 9).
- Running the rollback itself in production, or changing deploy pipelines.
- Runtime data: the skill reads code and diffs, not live databases or topics.
- Replacing human review: it informs the reviewer; it doesn't approve.

02 · User story

Know before you merge whether you can roll back

Primary user story

As a developer raising a pull request that changes a database migration or an event contract, I want GitHub Copilot to tell me before I merge whether the change can be rolled back, and if not, how to split it, so that a rollback never makes an incident worse.

Supporting stories

Priya
Payments engineer · author of PR #482

As the author, I want **a plain verdict and the reason, in my editor**, so that I can fix the change before anyone reviews it.

Sam
Tech lead · reviewer

As a reviewer, I want **every reader of a changed column or field, including ones outside the PR**, so that I review the blast radius, not just the diff.

Aisha
On-call engineer · reliability

As on-call, I want **a safe rollback window for every release**, so that I know whether pressing rollback will help or hurt.

Tom
Finance systems · reconciliation owner

As the owner of a downstream job, I want **changes to data I read to be flagged before merge**, so that my totals never break or drift silently.

Acceptance criteria

1. **Given** a PR with a destructive migration, **when** someone asks "can we roll back?", **then** the answer starts with **Verdict: ONE-WAY** and a safe rollback window.
2. **Then** each finding says when the door closes, what breaks on rollback (quoting the real error when known), and what data is lost.
3. **Then** the blast radius lists readers outside the PR, marked **not in this PR**.
4. **Then** it proposes an expand → migrate → contract → drop split, justifies each step, and generates it **only when asked**.
5. **Given** a safe change, **then** the verdict is **TWO-WAY**, window unlimited, with what was checked.
6. **Given** no Node.js, **then** it still reviews from the catalogue and says the scripts didn't run.

Journey

Stage	Today	With One-Way Door
Write	Sensible refactor, tests green	**Ask Copilot: ONE-WAY, three findings**
Review	Approved on the diff	**Reviewer sees reconciliation-batch**
Deploy	One step, all at once	**Four steps, each reversible**
Roll back	Old code won't start; hotfix	**Previous step runs fine**
Aftermath	50 rows at £0.00, no alert	**Totals match at every stage**

03 · Skill name and mission · 3.1

One skill, one question, answered before merge

Skill

Name	`one-way-door`
Location	`.github/skills/one-way-door/`
Type	GitHub Copilot agent skill (`SKILL.md`)
Runs in	VS Code agent mode, Copilot CLI, Copilot cloud agent, Copilot code review
Needs	git; Node.js 20+ for its scripts (optional)
Licence	MIT
Version	1.0.0 (plugin manifest)

Mission

For every pull request, answer the question no pipeline asks: *if this goes wrong after deploy, can we roll back?* Say it plainly, before merge, inside the tools engineers already use, and show the safe way to make the same change.

What it does

1 · Classify

Every change is **two-way** (safe to roll back) or **one-way** (not safe).

2 · Explain

For each one-way change: **when the door closes**, **what breaks** on rollback, and a **safe rollback window**.

3 · Blast radius

Finds **other code** reading the same table, column, topic or field, and marks what isn't in the PR.

4 · Split

Proposes **expand → migrate → contract → drop**, and generates it as branches when asked.

How Copilot decides to load it: the description (its only trigger)

```
Rollback-safety review for pull requests, diffs and releases. Answers "if this goes wrong after deploy, can we roll back?" Use when reviewing a PR, checking deploy or release readiness, asking "is this safe to ship" or "can we roll back", or when a change touches database migrations (Flyway, Liquibase, EF Core), event or message schemas (Kafka, JSON Schema, Avro, Service Bus), serialised DTOs, caches, idempotency keys, partner webhooks or API versions. Classifies each change as one-way or two-way, says when the door closes and what breaks on rollback, gives a safe rollback window, finds other code that reads the same table, column or topic, and proposes an expand, migrate, contract, drop split it can generate as branches.
```

03 · Skill name and mission · 3.2

Components and design principles

Components

Part	Path	Role
Skill instructions	`.github/skills/one-way-door/SKILL.md`	The 8-step procedure, gotchas and troubleshooting (167 lines)
Pattern catalogue	`references/patterns.md`	24 one-way patterns in 4 buckets, each with its safe split
Output contract	`references/output-format.md`	The exact PR comment shape, with worked examples
Migration scanner	`scripts/scan-migrations.mjs`	Destructive DDL in Flyway, Liquibase and EF Core migrations
Schema checker	`scripts/schema-compat.mjs`	Breaking JSON Schema changes, in the old reader's direction
Path rule (optional)	`.github/instructions/migrations.instructions.md`	Rollback rules applied whenever Copilot touches migrations or contracts
Hook (optional)	`.github/hooks/one-way-door.json`	preToolUse: blocks agent edits to migrations already on the default branch
Agent environment	`.github/workflows/copilot-setup-steps.yml`	Node 20 and the default branch, for the cloud agent and code review
Plugin	`plugins/one-way-door/`	Skill and hook as one install, generated and kept in sync by a test

Design principles

Scripts check, Copilot judges

Scripts find the mechanical facts. Copilot judges who else reads the data and what old code does with it.

Copilot's own formats

Documented skill, instruction and hook formats. No custom runtime: every surface reads it.

Offline and dependency-free

git and Node's standard library. Nothing to install, call or keep running.

Load only what's needed

Description first; then the 167-line `SKILL.md`; references only when needed.

Plain, fixed verdicts

ONE-WAY or TWO-WAY, a window and reasons, in a fixed shape. Real errors, no hedging.

Safe by default

Never changes code unasked. Unreachable readers are assumed to exist.

Output contract

```
## One-way door check
**Verdict: ONE-WAY | TWO-WAY** · Safe rollback window: **none (…) | until <event> | unlimited**
### Findings        1. Title · object · file:line · pattern ids  →  Door closes · If rolled back · Data loss
### Blast radius    | Reader | Where | Uses | Breaks when |   (readers outside the PR marked "not in this PR")
### Suggested split | 1. Expand | 2. Migrate | 3. Contract | 4. Drop |  + offer to generate
<sub>scan-migrations: n findings · schema-compat: n findings · checked base...head</sub>
```

03 · Skill name and mission · 3.3

How it works: scripts check, Copilot judges

Prompt "can we roll back?" › **Match** description only › **SKILL.md** loads the procedure › **Two scripts** git diff, offline › **Judgement** readers, old vs new › **PR comment** fixed format

The procedure (SKILL.md)

#	Step	What happens
1	Work out base and head	Default branch from `origin/HEAD`; always diff from the merge base (`base ... head`, three dots).
2	Bucket the files	Data, Messages, State or External, by path and content.
3	Run both scripts	Mechanical findings as JSON. Without Node, step 4 does the work alone.
4	Read the patterns	Match each change to the catalogue, including buckets the scripts don't cover.
5	Find the blast radius	Search the whole repo (base and head) for every reader; mark those the PR doesn't change.
6	Old code against new state	What does the previous release do with the new table, events or cache? Quote the real error.
7	Write the comment	The verdict line, findings, blast radius and split, in the fixed format.
8	Generate the split	Only when asked: stacked branches, each commit saying why it rolls back.

`scan-migrations.mjs`

In: `--base --head [--repo] [--worktree]`. **Reads:** Flyway SQL, Liquibase (XML, YAML, formatted SQL), EF Core. **Finds:** column and table renames and drops, retypes, NOT NULL, in-place conversion, backfill, row deletes, edited applied migrations.

`schema-compat.mjs`

In: `--base --head [--dir]… [--worktree]`. **Reads:** JSON Schema (`*.schema.json`). **Finds:** fields removed, renamed, retyped or made optional, enum values added, new required fields, strict consumers, missing primitive defaults, deleted schemas.

Both: Node 20+ standard library, no network, read-only git; JSON `findings[]` (file, line, pattern, object, severity, detail); always exit 0.

Pattern catalogue: 24 one-way doors

Data · 10

- `column-rename`
- `table-rename`
- `column-drop`
- `table-drop`
- `column-retype`
- `in-place-conversion`
- `not-null` `backfill`
- `row-delete`
- `applied-migration-edited`

Messages · 9

- `event-field-removed`
- `event-field-renamed`
- `event-field-retyped`
- `event-field-made-optional`
- `event-enum-value-added`
- `event-required-field-added`
- `strict-consumer-new-field`
- `missing-primitive-default`
- `event-schema-deleted`

State · 2

- `cache-format-change`
- `idempotency-key-format-change`

External · 3

- `servicebus-entity-deleted`
- `apim-version-removed`
- `webhook-payload-change`

04 · Repository link

Where the code lives

Repository

TBD

To be confirmed when the repository is published to GitHub. Once it is, the plugin installs with `copilot plugin install OWNER/REPO:plugins/one-way-door`.

TBD

Repository layout

```
.github/
  skills/one-way-door/      the deliverable
    SKILL.md
    references/             patterns.md, output-format.md
    scripts/                two .mjs scripts + tests
  instructions/             migrations.instructions.md
  hooks/                    one-way-door.json + .sh/.ps1
  plugin/marketplace.json
  workflows/                ci, laptop-demo, setup steps
plugins/one-way-door/       generated plugin
contracts/events/           PaymentSettled schema
demo/
  settlement-service/       Spring Boot producer
  ledger-service/           Spring Boot consumer
  reconciliation-batch/     finance job
  control-centre/           Java + React console
  start.sh  start.cmd
  scripts/                  Docker alternative
docs/                       setup, demo script, results
tests/                      repo consistency tests
README.md  DEMO.md  LICENSE
```

Branches

Branch	Purpose
`main`	v1: the skill, demo and docs
`pr-482-minor-units`	The one-way PR: V3 renames and converts
`split/1-expand`	V3 adds `amount_minor`
`split/2-migrate`	V4 backfills and checks
`split/3-contract`	V5: readers switch; trigger
`split/4-drop`	V6 drops `amount`

Quality gates

Check	Covers
ci.yml	62 skill, hook and plugin tests on Linux, Windows and macOS; service unit tests
laptop-demo.yml	Headless incident and rollback matrix on Windows and macOS; UI bundle is current
Consistency	SKILL.md rules, catalogue coverage, `applyTo` coverage, plugin in sync
Control Centre	6 JUnit tests for the rules ported from the scripts

05 · Setup instructions · 5.1

Tech stack and install options

Tech stack

Component	Version	Needed for	Notes
git	2.x	Everything	With the base branch's history (not a shallow clone), so the merge base exists.
Node.js	20 or later	The two scripts	Standard library only: no `npm install`, no network. Optional: without it the skill reviews from the catalogue.
bash 3.2+ / PowerShell 7+	—	The hook (optional)	bash on macOS, Linux and the cloud agent; `pwsh` on Windows.
GitHub Copilot plan	Free or paid	Everything	Free: VS Code agent mode and CLI only. Pro, Pro+, Max, Business, Enterprise: all four surfaces.
VS Code	1.109 or later	VS Code	Agent skills are on by default from 1.109. Workspace Trust required.
Copilot CLI	Current	Copilot CLI	`npm i -g @github/copilot` (Node 22+), WinGet or Homebrew. Windows needs PowerShell 6+.
GitHub repo + Actions	—	Cloud agent, code review	Both run on Actions runners; `copilot-setup-steps.yml` prepares Node and git.
Org policies	—	Business, Enterprise	"Copilot Agent Mode in IDE Chat", "Copilot CLI", "Copilot cloud agent" (off by default), "Copilot code review".

Not needed: a server, database, Docker, npm packages, an MCP server, network access, or any account beyond Copilot. The skill never builds or runs your code.

Install options

A · Copy into the repository (every surface)

```
cp -R $SRC/.github/skills/one-way-door  .github/skills/
cp $SRC/.github/instructions/migrations.instructions.md
   .github/instructions/                # optional rule
cp -R $SRC/.github/hooks/.  .github/hooks/   # optional
cp $SRC/.github/workflows/copilot-setup-steps.yml \
   .github/workflows/                   # cloud agent,
```

Then set `applyTo` to your migration paths and merge to the default branch.

B · Install the plugin (VS Code and CLI)

```
copilot plugin install OWNER/REPO:plugins/one-way-door
copilot plugin install /path/to/plugins/one-way-door
```

VS Code: set `chat.plugins.enabled`, then **Chat: Install Plugin From Source**, or add the folder to `chat.pluginLocations`. The cloud agent and code review don't load plugins: use option A there.

What works where	VS Code	Copilot CLI	Cloud agent	Code review
Copy into the repo	**yes**	**yes**	**yes**	**yes, "when relevant"**
Plugin	**yes**	**yes**	**no**	**no**
Hook	yes (preview)	**yes**	**yes (bash only)**	not documented

05 · Setup instructions · 5.2

Setup by Copilot surface

VS Code (agent mode)

1. VS Code 1.109+, signed in to Copilot; git and Node 20+ on the terminal's PATH.
2. Open the folder that contains `.github/`; **trust the workspace**.
3. Chat → **Agent**. Check `chat.useHooks` is on (default).
4. **Check:** type `/` or `/skills` and look for `one-way-door`; `/hooks` for the hook; **Developer: Open Agent Debug Logs** if it's missing.
5. Ask; approve the `git` and `node` commands. References should list `SKILL.md`.

Copilot CLI

1. Install: `npm i -g @github/copilot`, `winget install GitHub.Copilot` or `brew install --cask copilot-cli`.
2. Sign in with `/login` (or a fine-grained token with "Copilot Requests").
3. Run `copilot` in the repo root and trust the folder.
4. **Check:** `/skills list` shows `one-way-door`.
5. Ask; approve the commands. A denied hook call shows inline with its reason.

Copilot cloud agent

1. Paid plan. Org admin: policy **"Copilot cloud agent"** → Enabled (off by default), then Cloud agent → Repository access.
2. Option A files on the **default branch**, including `copilot-setup-steps.yml` (job name `copilot-setup-steps`).
3. Trigger: comment `@copilot Review this PR for rollback safety`, assign an issue, or use the agents panel.
4. **Check:** the session log shows `SKILL.md` read and both scripts run. Firewall doesn't affect them: they are offline.

Copilot code review

1. Paid plan. Org admin: policy **"Copilot code review"**; Actions runners available.
2. Option A files on the PR's **head branch**: code review reads skills from there.
3. One PR: Reviewers → **Copilot**. Every PR: a ruleset with **"Automatically request Copilot code review"**.
4. Expect the skill to be used "when relevant". Whether code review runs its scripts or hooks isn't documented; for a guaranteed full review, use the cloud agent.

Verify, then troubleshoot

```
node .github/skills/one-way-door/scripts/scan-migrations.mjs --base main --head my-branch   # JSON, exit 0
node .github/skills/one-way-door/scripts/schema-compat.mjs  --base main --head my-branch
```

Symptom	Fix
Skill not listed	`name:` must equal the folder name; open the repo root; trust the workspace; check policies.
`unknown ref` in script output	`git fetch origin main:main`, or pass `--base origin/main`; unshallow the clone.
Every tool call denied (CLI, Windows)	The hook's shell is missing or policy blocks unsigned scripts: install PowerShell 7, or remove the hook.
"Scripts didn't run" in the comment	Install Node 20+. The review is still produced from the catalogue.

Full guide with sources: `docs/setup.md` (checked against the live GitHub and VS Code docs on 13 October 2026).

06 · Demo instructions · 6.1

Set up and start the demo: Java and git, no Docker

What runs (one command, one Java process)

Browser React UI · :7070 › **Control Centre** Spring Boot · :7070 › **PostgreSQL 16** embedded · :55432 + **Kafka (KRaft)** in the JVM · :9092 › **3 services** built from git

Service	Role in the story
settlement-service (:8081)	The writer: `POST /settlements` → `PaymentSettled` event on Kafka
ledger-service (:8082)	The reader: consumes events, books `ledger_entries`; Flyway migrations, Hibernate validation
reconciliation-batch	The reader nobody remembered: finance's `SUM(amount)`, straight from Postgres

You need

Need	Detail
Java (JDK)	21 or later (tested on 21 and 25)
git	2.x, and the six demo branches locally
Internet, once	Maven Central or a company mirror
Disk, ports	~2.5 GB; 7070, 8081, 8082, 9092, 55432
Not needed	Maven, Docker, Postgres, Kafka, npm, admin rights

Maven and Node libraries

- **Maven:** each project's wrapper (`mvnw` / `mvnw.cmd`) downloads Maven 3.9.11 and ~400 MB of libraries into `~/.m2` on first build. Mirrors via `settings.xml`; `MVNW_REPOURL` for Maven itself.
- **Node:** the skill's scripts need no packages. The UI is committed pre-built; `npm ci` only to change it.

Steps

```
# 1. Clone or unzip, then recreate the branches
git clone <repo-url> oneway-door && cd oneway-door
sh demo/restore-branches.sh  # macOS / Linux
demo\restore-branches.cmd    # Windows

# 2. Start (builds itself the first time)
demo/start.sh            # macOS / Linux
demo\start.cmd           # Windows

# 3. Optional: download everything ahead
demo/start.sh --owd.run=prepare
```

In the browser (http://localhost:7070)

1. **System check:** Java, git, branches, Postgres + Kafka green.
2. **Prepare:** builds every branch, starts v1, sends 10 settlements (~15 s once built).
3. The map shows both services on **v1 · main**, UP. You're ready.

Stop with Ctrl+C: services, Kafka and Postgres stop in about 2 s. Full guide: `DEMO.md`.

06 · Demo instructions · 6.2

Architecture: the demo system

Northwind Payments is invented. Three small Spring Boot services, one Kafka topic and one Postgres table: just enough to show a rollback that can't work.

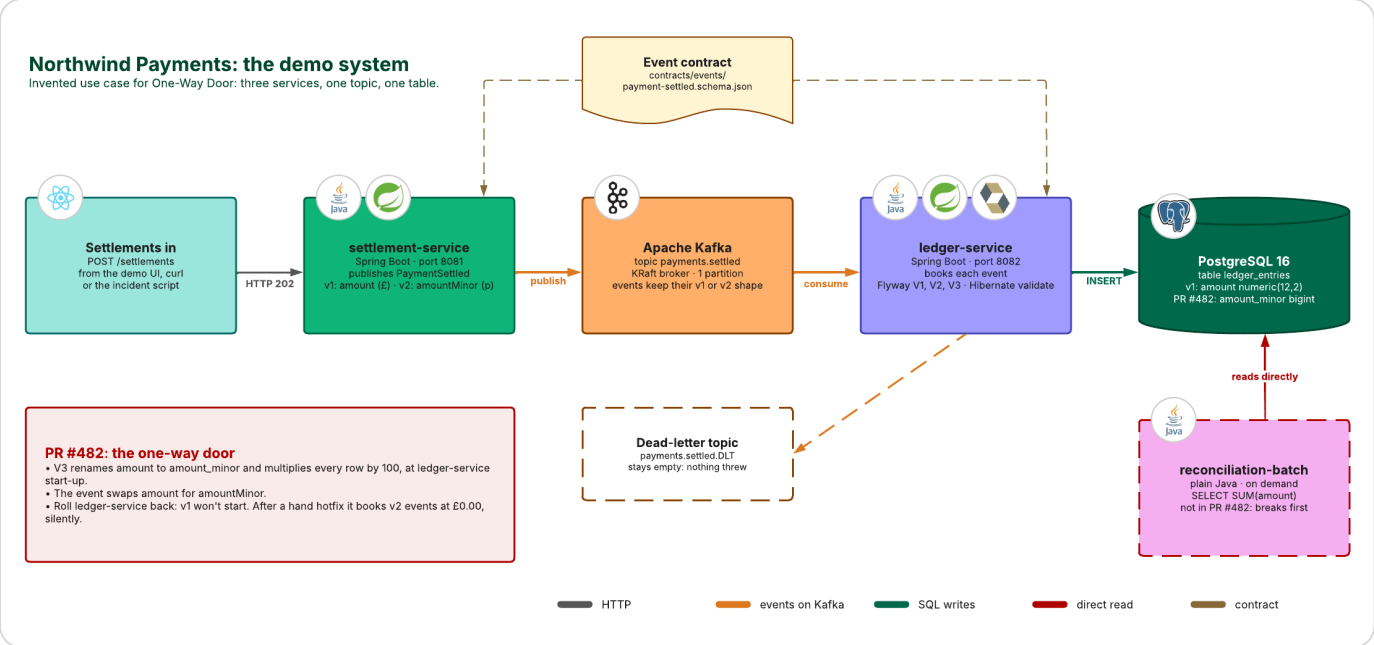


Figure 1. The demo system, drawn in Lucidchart. Badges show the stack: Java 21, Spring Boot, Hibernate, Apache Kafka, PostgreSQL 16 and the React UI.

The writer

settlement-service takes `POST /settlements` and publishes `PaymentSettled`. v1 sends `amount` in pounds; PR #482 sends `amountMinor` in pence.

The reader

ledger-service consumes each event and books one row in `ledger_entries`. Flyway owns the schema; Hibernate validates it at start-up, so a wrong schema fails loudly.

The reader nobody remembered

reconciliation-batch runs finance's `SUM(amount)` straight from Postgres. It isn't in PR #482, so it breaks first. This is the blast radius.

Kept real on purpose	Why it matters to the story
Type headers off	The producer sends no Java type; the consumer maps by field name. So v1 and v2 events are both readable by both versions, as in most real estates.
Jackson defaults	Unknown fields are ignored. That default is why v1 books a v2 event at £0.00 without an exception or a dead letter.
`ddl-auto=validate`	v1 ledger-service refuses to start against the V3 schema: `missing column [amount]`.
Real rollbacks	A rollback runs the previous commit's code, built from a git worktree, against the current database and topic.

06 · Demo instructions · 6.3

Architecture: the demo runtime on one laptop

The org laptop has Java and git, and no Docker. So one Java process, the Control Centre, runs the infrastructure itself and starts the real services as child processes.

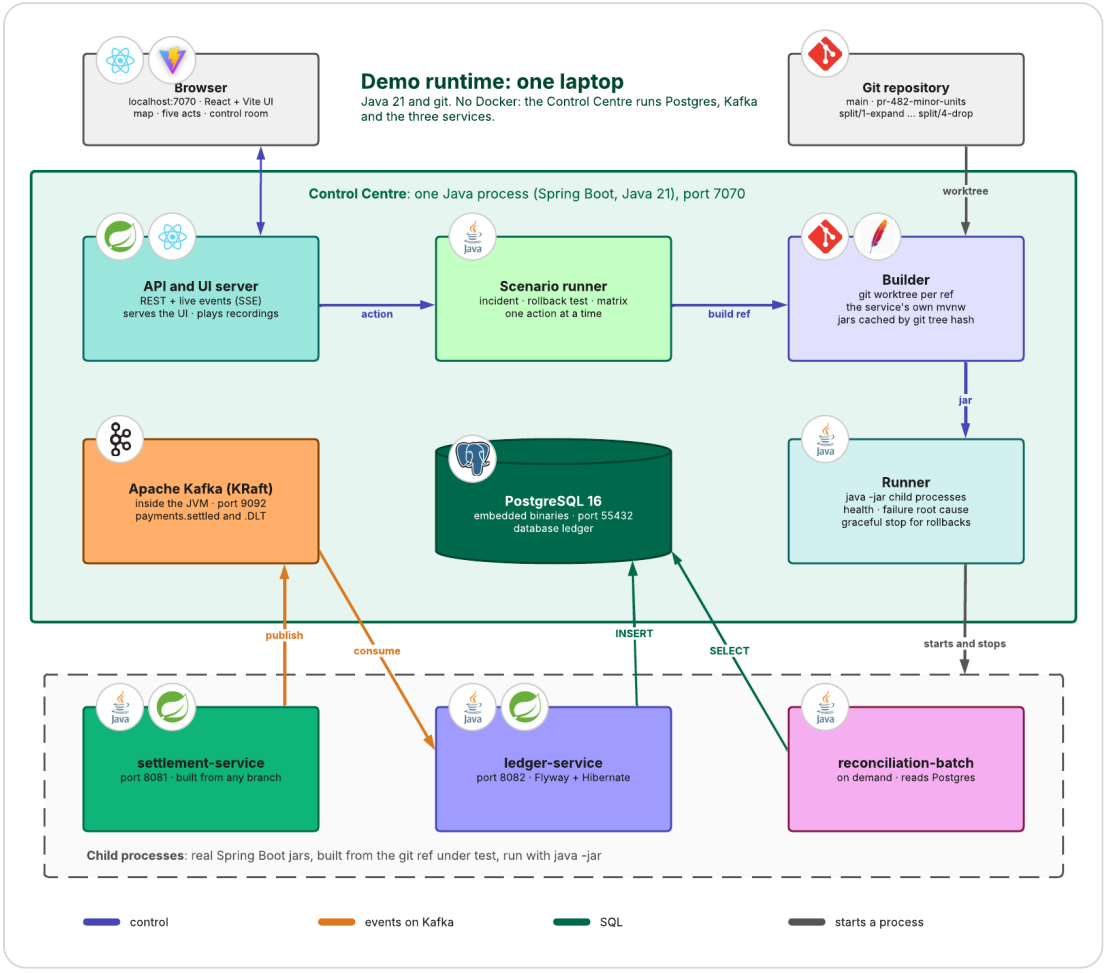


Figure 2. The demo runtime, drawn in Lucidchart. One command starts it: `demo/start.sh` or `demo\start.cmd`.

Part	What it is	What it does in the demo
API and UI server	Spring Boot on :7070; React + Vite UI, built in	REST for each action; live events to the browser over SSE; plays the recorded run in Replay mode
Scenario runner	Incident, rollback test, matrix	One action at a time; the same steps and checks as the scripts in `demo/scripts`
Builder	git worktree per ref, the service's own `mvnw`	Builds any branch; caches jars by git tree hash, so a rebuild is instant
Runner	`java -jar` child processes	Health checks, start-up failure root cause, graceful stop so a rollback frees the Kafka group at once
PostgreSQL 16	Embedded binaries, :55432	Database `ledger`; stopped cleanly on exit
Apache Kafka	KRaft broker in the JVM, :9092	Topics `payments.settled` and `.DLT`; an observer reads each event's real shape for the map

06 · Demo instructions · 6.4

Run the five acts in four minutes

Act	Time	Click and show	Say
1 · The PR	0:35	Tab 1: the V3 migration and event change; **Run tests** (green). Send the Copilot prompt in VS Code now.	"PR #482 moves to pence. Tests are green. You'd approve this."
2 · The incident	1:10	Tab 2: **Start**, then → for 7 steps. The controls and live numbers sit under the map.	One line per step, ending: "Every dashboard is green, and fifty settlements are booked at £0.00."
3 · The review	1:00	VS Code: Copilot's answer. Tab 3: each finding beside the real error from act 2.	"One folder in .github/skills. Scripts check, Copilot judges."
4 · The split	0:35	Tab 4: expand, migrate, contract, drop.	"Same destination, four steps, each reversible."
5 · The proof	0:40	Tab 5: matrix 4 PASS, 1 FAIL. Optionally **Run this row** live.	"PR #482 fails before anyone presses rollback."

Act 2, step by step (what each Java service really does)

Time	Step	What happens
16:30	v1 in pounds	10 settlements published with `amount`; ledger books them in `numeric(12,2)`.
16:40	PR #482 ships	ledger v2 starts; Flyway runs V3 (rename, ×100) in 5 ms; settlement v2 sends `amountMinor`.
16:55	Reconciliation fails	`ERROR: column "amount" does not exist`, in a job nobody changed.
16:58	Roll back	v1 ledger stops: `Schema-validation: missing column [amount]`. Flyway has no undo.
17:00	Backlog builds	50 v2 events queue on Kafka; nobody is consuming.
17:25	Hand hotfix	DBA reverses V3; v1 starts and drains the queue.
17:26	£0.00	Jackson ignores `amountMinor`; missing `double amount` = 0.0. 50 rows at £0.00, DLT empty.

The day before

- System check green; Prepare
- Run act 2 once
- Control room → **Record the incident**
- Act 5 → Run all 5 rows
- Copilot answers in VS Code

Fallbacks

- **Replay** toggle: plays the recorded real run, labelled as a recording
- Copilot slow: tab 3 shows the same review
- Last resort: the backup video

Headless checks

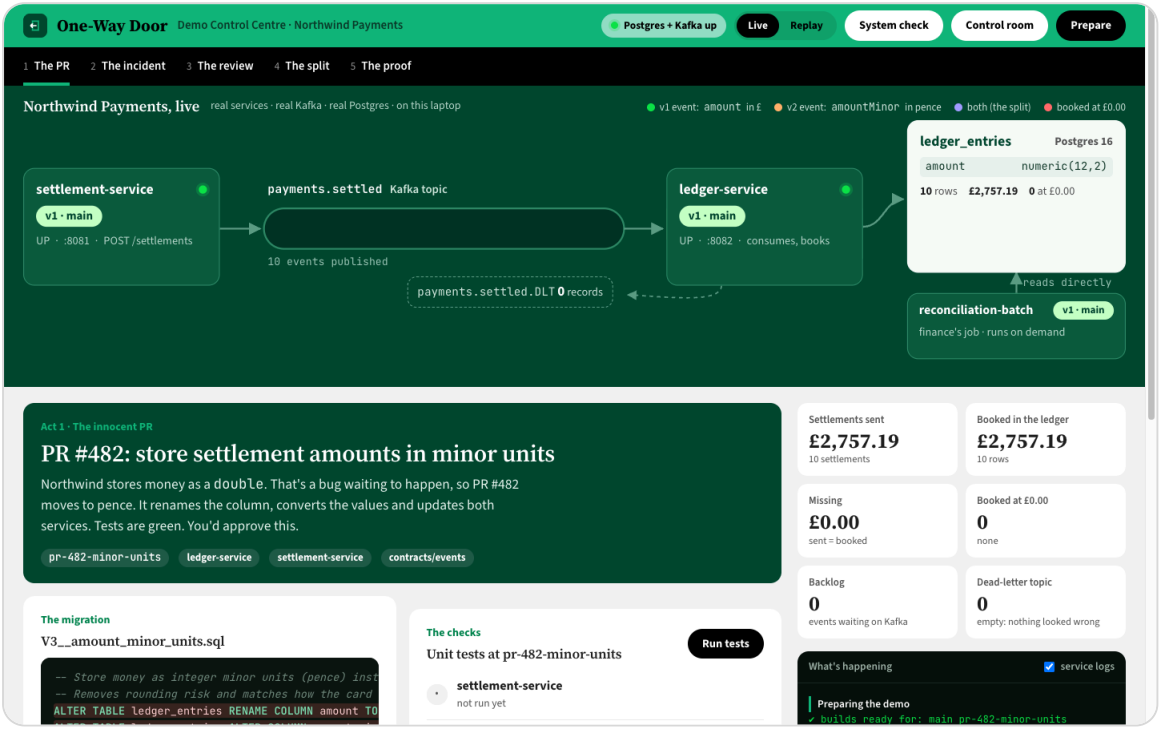
```
demo/start.sh --owd.run=incident
demo/start.sh --owd.run=matrix
```

Exit 0 = reproduced; 4 PASS + 1 FAIL.

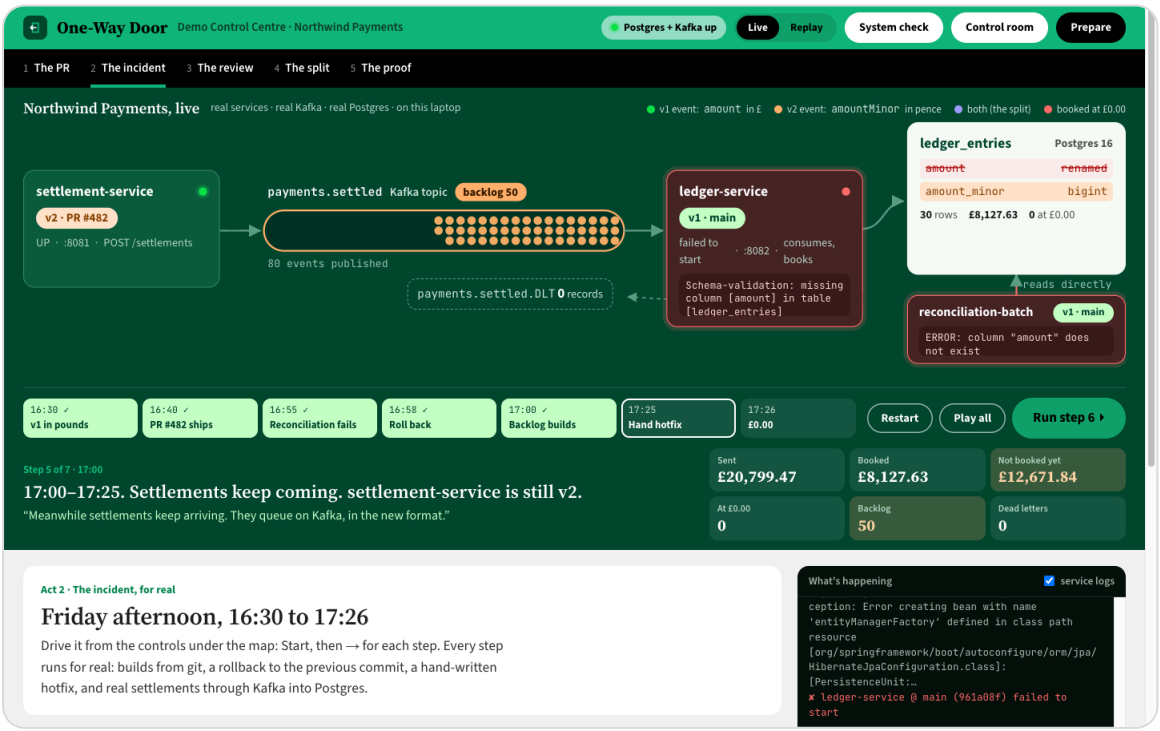
06 · Demo instructions · 6.5

The Control Centre: the PR and the rollback

Screens from a live run on 14 October 2026: real services, real Kafka, real Postgres, on one laptop. Amounts are random each run; the outcome is the same every time.



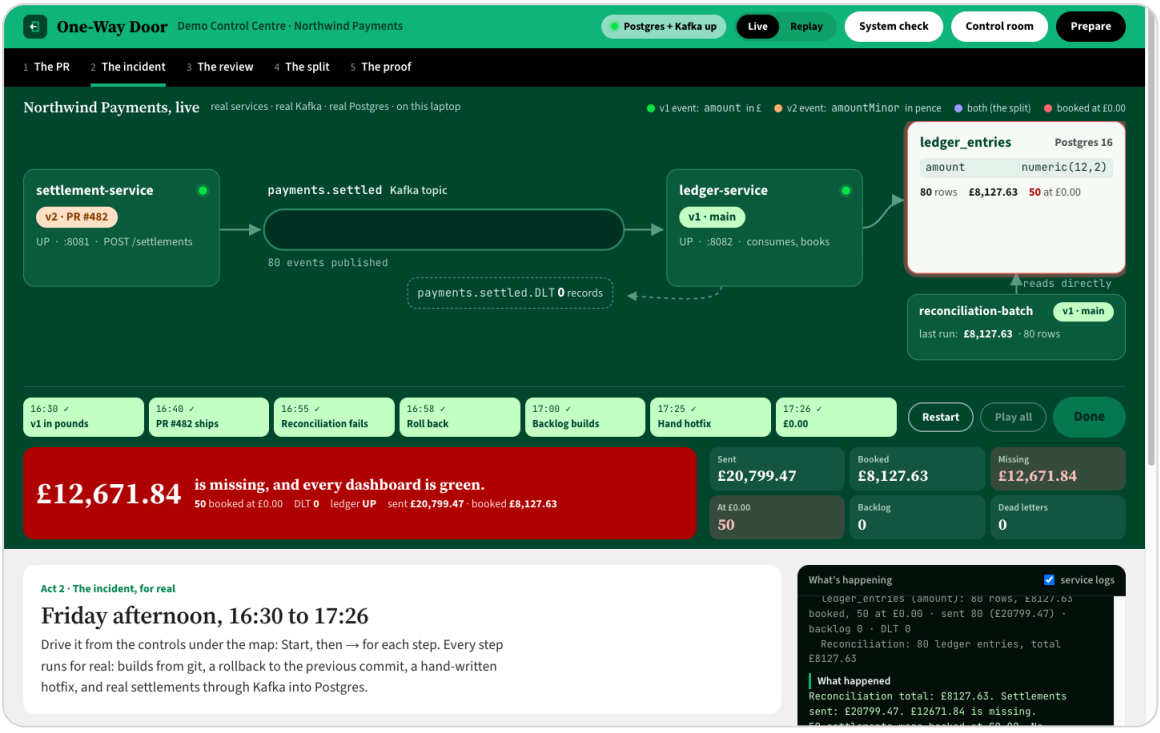
Act 1 · The PR. Both services on v1; the table has `amount numeric(12,2)`. Below the map: PR #482's migration and its passing tests.



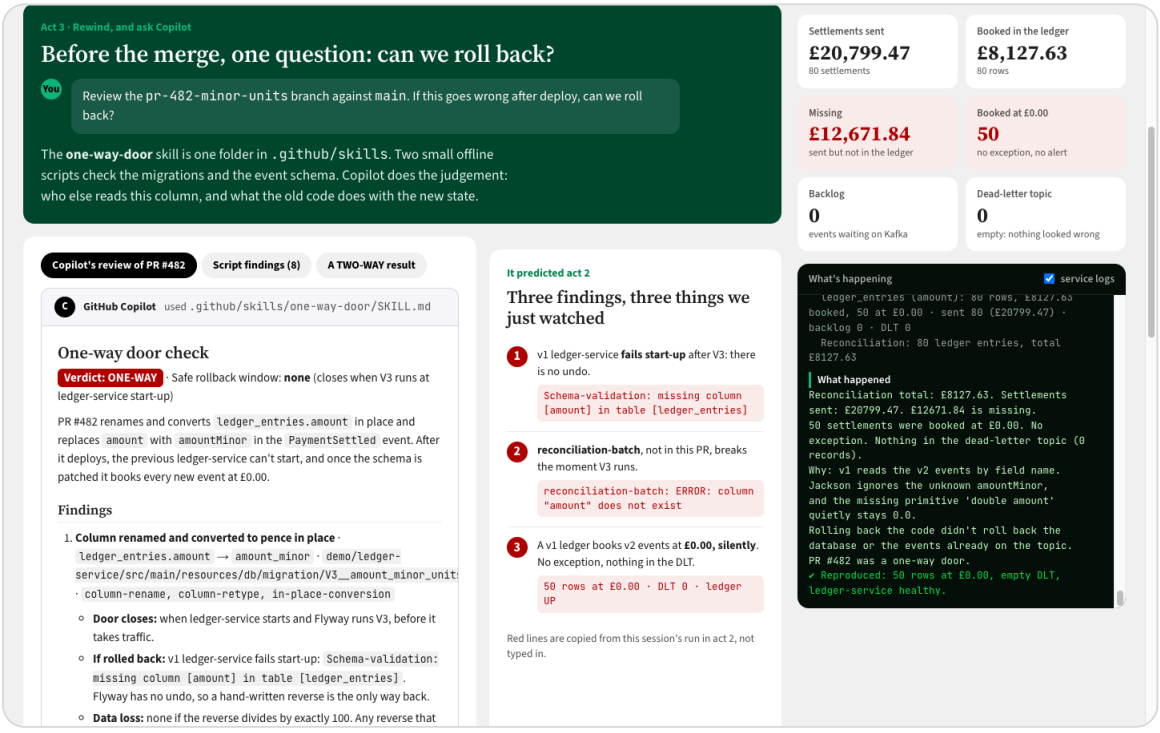
Act 2 · 17:00. Rolled back, v1 ledger-service won't start: `missing column [amount]`. Reconciliation fails too. 50 events wait on Kafka; £12,671.84 is not booked yet.

06 · Demo instructions · 6.6

The Control Centre: £0.00, and the review



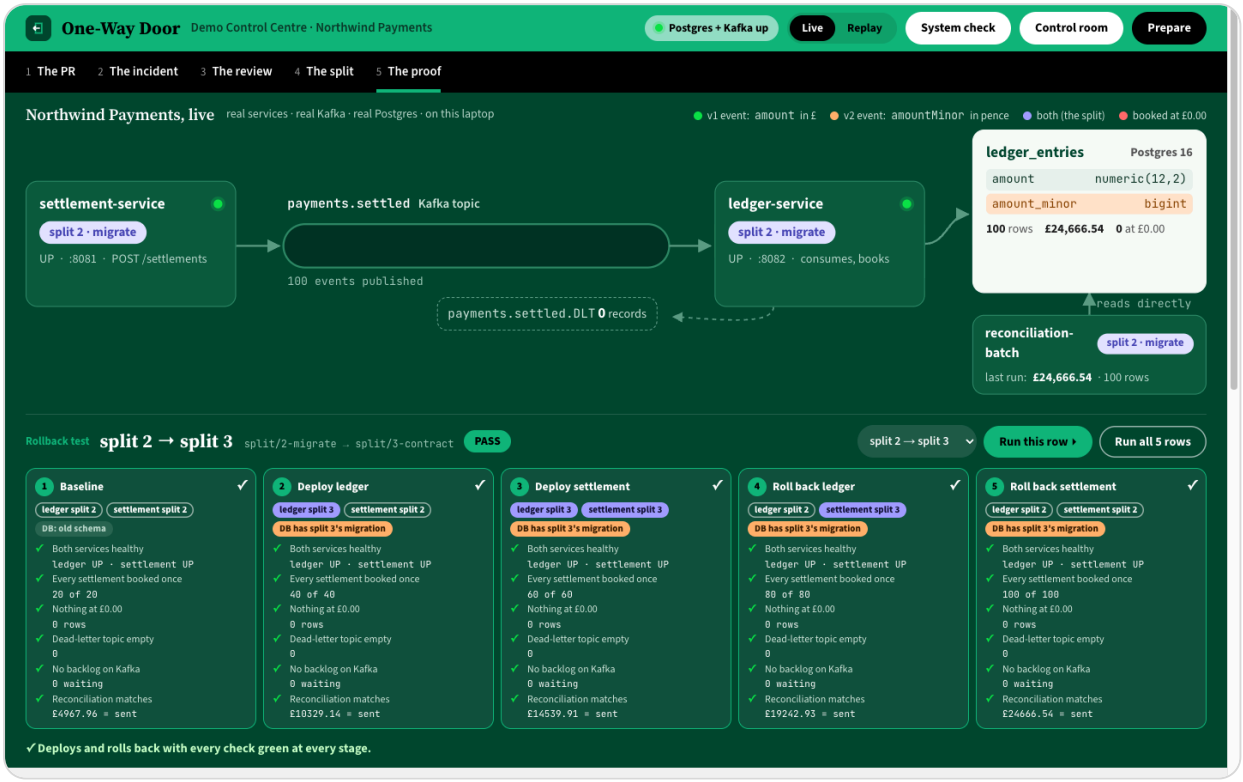
Act 2 · 17:26. After the hand hotfix, v1 drains the queue. £12,671.84 is missing and 50 rows are at £0.00. The DLT is empty and every service is UP.



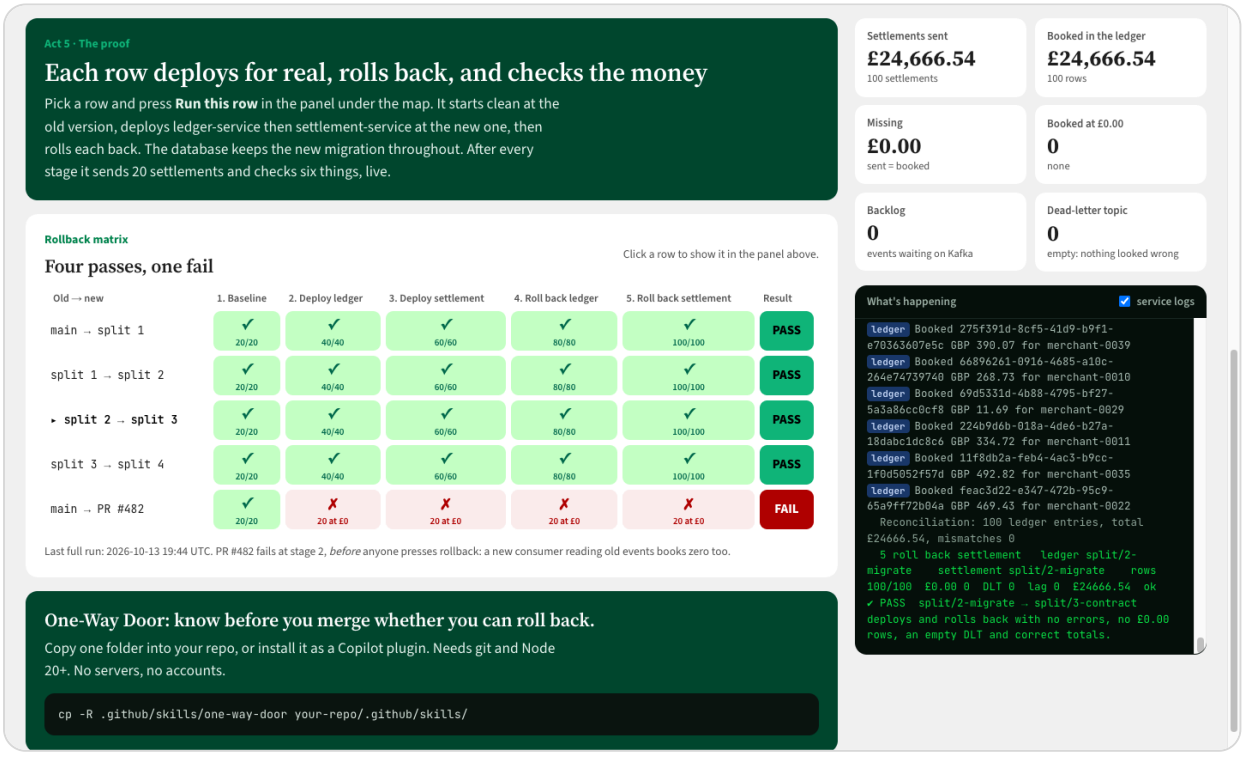
Act 3 · The review. The skill's verdict, ONE-WAY with no safe window, beside the three things it predicted. The red lines are copied from act 2's run, not typed in.

06 · Demo instructions · 6.7

The Control Centre: the proof, live



Act 5 · One row, run live. `split/2-migrate → split/3-contract`: deploy ledger, deploy settlement, roll each back. After every stage, 20 settlements and six checks. PASS in about 20 seconds.



Act 5 · The rollback matrix. Each split step deploys and rolls back cleanly. PR #482 fails at stage 2, before anyone presses rollback: a new consumer reading old events books £0.00 too.

07 · Before and after completions

What Copilot says without the skill, and with it

Same prompt, same pull request: *"Review the pr-482-minor-units branch against main. If this goes wrong after deploy, can we roll back?"*

Before · without the skill

Looks good overall. Storing money as integer pence avoids floating-point rounding, which is a good improvement.

- The migration converts existing values with `* 100`; consider a test for the conversion.
- Both services are updated consistently.
- To roll back, revert the commit and redeploy.

Illustrative: a typical generic review. It answers the code, not the question. No verdict, no window, no other readers, and "revert and redeploy" is exactly what fails.

After · with One-Way Door

Verdict: ONE-WAY · Safe rollback window: **none** (closes when V3 runs at ledger-service start-up)

Findings

1. **Column renamed and converted in place** · `V3…sql:3` · rename, retype, in-place-conversion. If rolled back: v1 fails start-up, `missing column [amount]`.
2. **amount replaced by amountMinor** in the event · door closes with the first v2 event.
3. **Missing amount silently becomes £0.00** · no exception, nothing in the DLT.

Blast radius

reconciliation-batch (not in this PR): `SUM(amount)` breaks when V3 runs.

Suggested split

Expand → migrate → contract → drop, each step justified. Reply "generate the split".

From the skill's worked example (`references/output-format.md`); scripts: 3 + 5 findings.

Before and after in the running system (measured in the demo)

Moment	PR #482 merged as written	The split, as the skill proposes
Deploy the new reader	20 old events booked at £0.00	Reads either field; all booked correctly
Finance reconciliation	Fails: column "amount" does not exist	Totals match at every stage
Roll back the reader	Old code won't start; hand hotfix	Previous step runs on the new table
Money	£12,708.11 missing, no alert (recorded run)	£0 missing; DLT empty; no backlog
Rollback matrix	FAIL at stage 2	PASS: 4 of 4 steps, 5 stages each

Before merge
when the problem is found, instead of at 16:55 on a Friday

3 of 3
findings that happened in the live incident, as predicted

1 reader
outside the PR found: the job that broke first

08 · Reuse statements

Built for any repository, not just the demo

Reuse statement

One-Way Door is generic. Nothing in `SKILL.md` or its scripts refers to the demo; a test enforces that. Any team can reuse it in any GitHub repository by copying one folder or installing one plugin, with no servers, dependencies or changes to their build.

Clear statements

1. **Reusable as it is** in any repository with Flyway, Liquibase or EF Core migrations, or JSON Schema event contracts.
2. **Language-agnostic.** Blast radius uses `git grep`, so readers in Java, C#, Python, TypeScript or SQL are all found.
3. **Works without its scripts.** Avro, Protobuf, caches, webhooks and APIs are reviewed from the catalogue.
4. **No infrastructure.** git and, optionally, Node 20+. No packages, network or accounts beyond Copilot.
5. **Every Copilot surface** from one copy: VS Code, CLI, cloud agent and code review.
6. **Optional parts are independent.** Take the skill alone, or add the rule and the hook.
7. **MIT licensed.** Copy, adapt and redistribute.
8. **The demo is reusable too** as a pattern: build each version from git, mix old and new, check the money.

What to reuse, and how

Asset	Reuse	How
The skill	**As it is**	Copy `.github/skills/one-way-door/`, or install the plugin.
Path rule	Configure	Copy, then set `applyTo` to your migration and contract folders.
Hook	**As it is**	Copy `.github/hooks/`. Base branch: `origin/HEAD`, or set `ONE_WAY_DOOR_BASE`.
Agent environment	Merge	Add its two steps (Node 20, fetch default branch) to your `copilot-setup-steps.yml`.
Schema folders	Configure	`--dir` per folder; the skill finds them with `git ls-files '*.schema.json'`.
Pattern catalogue	Extend	Add a pattern with its safe split; a test checks every script pattern is documented.
Scripts	Extend	New formats (Avro, Protobuf) as new scripts: same contract, JSON out, exit 0.
Control Centre	Pattern	Demo-specific, but its rollback matrix shows how to prove any split.

Reuse in three steps

1. Copy the folder (or install the plugin).
2. Set `applyTo`; merge.
3. Ask "can we roll back?" on your next PR.

Versioning

Plugin manifest `1.0.0`. The plugin is generated from `.github` and a test fails if they drift. Reinstall plugins after updates.

Planned extensions

Readers across repositories (`find-readers`), rule-data checks, Spanner and BigQuery schemas.

09 · Known limitations

What it doesn't do yet, and what to do about it

#	Limitation	Impact	Mitigation
1	Scripts cover SQL (Postgres, MySQL, SQL Server dialects), Liquibase, EF Core and JSON Schema only	Avro, Protobuf, caches and external contracts get no mechanical check	The catalogue and Copilot's judgement cover them; add scripts with the same contract
2	One repository at a time	Readers in other repos are missed	Planned `find-readers` across related repos; it will assume unreachable repos read the data
3	Judgement comes from a language model	Answers can vary; it may miss a reader or over-flag	Scripts anchor the facts; fixed format; humans still review
4	Reads code and diffs, not runtime state	Doesn't know deploy order, flags or data volumes unless told	Say them in the prompt; the window uses "until <event>" when order holds the door open
5	Copilot code review uses skills "when relevant"	May not pick the skill; running scripts or hooks there is undocumented	Ask `@copilot` (cloud agent) for the full review
6	The split is proposed, not proven, by the skill	Generated branches need review and testing	The demo's rollback matrix shows how to prove each step
7	Needs the base branch's history	Shallow clones have no merge base	`git fetch --unshallow`; the setup steps fetch it for the cloud agent
8	Hooks: VS Code is preview; Windows PowerShell hook not yet run on real Windows	The guard may not fire on some setups	CI now runs the hook tests on Windows; the skill doesn't depend on the hook
9	Plugin hook relies on `${PLUGIN_ROOT}`	If Copilot CLI doesn't expand it, the plugin's hook allows everything	Use the in-repo hook (install option A)
10	Windows: a crashing hook blocks every tool in Copilot CLI	No PowerShell 7, or signed-scripts-only policy, stops Copilot working	Check `Get-ExecutionPolicy -List`; leave the hook out if needed
11	Depends on Copilot plans and org policies	Free has no cloud agent; the cloud agent is off by default in orgs	See section 5; ask the org admin
12	The demo Control Centre is for demonstrations	Embedded Postgres; fixed service ports; locked-down laptops may block the binaries	Replay mode needs nothing else; Docker scripts as an alternative

Still to verify by hand

- The Copilot test checklist (`docs/copilot-test-checklist.md`) on each surface with a real Copilot licence.
- VS Code showing a hook's deny reason in chat; the Windows hook end to end on the presenting laptop.

10 · Guardrails

What it will and won't do

The skill: reviews, never acts on its own

- Read-only by default. It never edits code, creates branches, pushes or opens pull requests unless the user asks (step 8).
- No pre-approved tools: `allowed-tools` is deliberately not set, so every command it runs needs the user's approval.
- States what it searched. Readers it can't check are assumed to exist, and it says so.
- Quotes real errors; marks when scripts didn't run; no guesswork words, no hype, British English.

The scripts: deterministic and offline

- Node standard library only; no packages, no network, no writes. They only read git.
- Same input, same JSON output. Always exit 0: findings are data, not failures.
- Always diff from the merge base, so other people's changes on main are never blamed on the PR.

The hook: a narrow guard that can't lock you out

- Denies only one thing: agent edits to migrations already on the default branch, with a reason and the next version to use.
- Reading, new migrations, repeatable and undo scripts and Liquibase changelogs stay allowed.
- Fails open on its own errors: a broken hook must never block every tool call.

The path rule: rollback rules where they matter

- Never rename, drop or retype a column in one step; never convert in place.
- Never edit an applied migration: add a new one.
- Event fields: add as optional; consumers move first; never let a missing number default to 0.

Platform controls respected

Workspace Trust, org and device policies, terminal approvals and the cloud agent's firewall all apply. The skill adds no way around them.

Data and privacy

No secrets or credentials needed. The scripts send nothing anywhere. Code is processed only by Copilot, under the user's existing licence.

Honest demo

Every rollback rebuilds the old code from git. Replay is labelled as a recording. All companies, people and amounts are invented.

Sign-off

Reviewed by: _____ Approved by: _____ Date: _____

Version 1.0